Argus — An Honest Autonomous AI Researcher

A 7×24 system that picks its own problems, writes the code, runs the experiments, and writes the paper — and tells you, without flinching, what it can and cannot do.

argusbot.cn · live dashboard

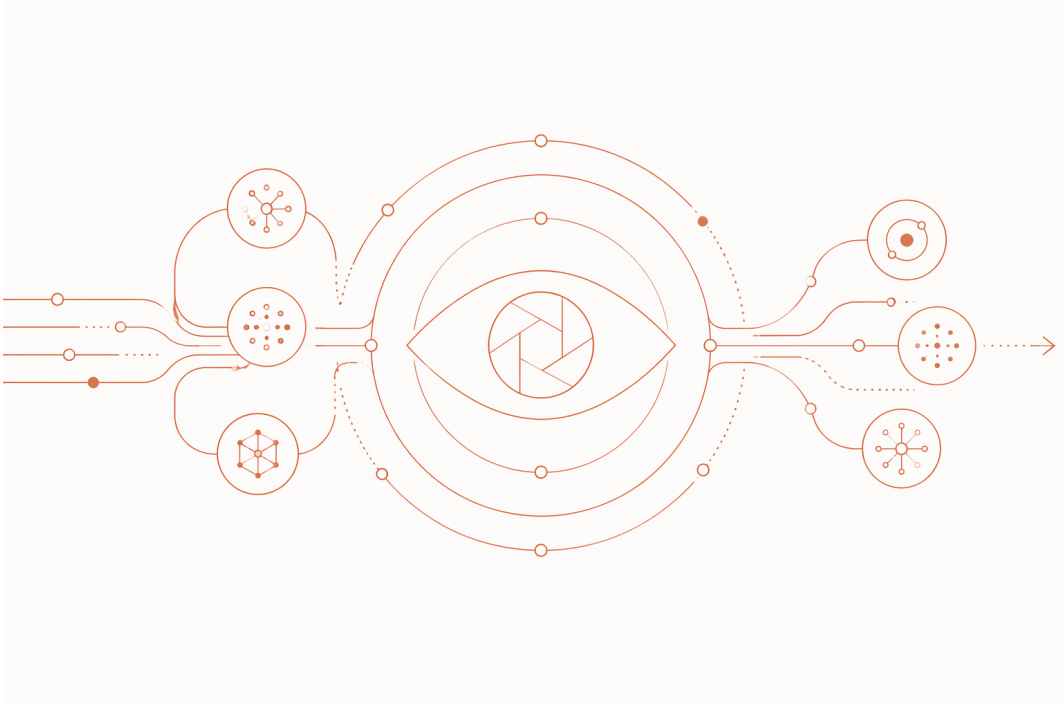
Why this post reads differently. Most "automated AI research" write-ups lead with a wall of state-of-the-art numbers. We are not going to do that, because it would be dishonest: our results are real but thin, hardware-caveated, and in one case within the confidence interval. What Argus actually demonstrates is a *system design* — a dumb harness, three adversarial agents, and a frozen definition of winning that the agents can never touch — and an unusually unvarnished account of where autonomous research currently breaks. The honesty is not a disclaimer bolted on at the end. It is the architecture.

1 Summary

Result	Headline	The honest caveat
nanochat 5-min invention	single-mechanism **0.9874** val BPB, below the same-protocol fair floor (0.9879)	the −0.0005 margin is *within* the 95% CI; the convincing −0.0024 comes from *stacking* mechanisms
End-to-end papers	2 compilable papers (11pp, 15pp) with figures + citations, 0 unresolved TODOs	the gates test *form*, not *truth*; the comparator anchors are mixed and the benchmark is synthetic
Tuner → inventor	first PASS-A: an argus-authored q/k `rotary`→`RMSNorm` seam beats the floor	a prior, timid invention closed as **PASS-B** — a measured negative

Every number was produced on **a single H100**, 10 seeds, no cherry-picking. Where a comparison crosses hardware (H100 vs B200) we say so and do not net the two.

2 The system: a dumb harness, three agents, the win defined outside



Argus is built on one conviction: **the harness is never smarter than the agent, and the definition of winning is always external.**

- **The harness is dumb on purpose.** It moves state through a stage machine, checks *form* (does the paper compile, does the run respect the budget), enforces gates and rollback — and never judges whether a contribution is *real*. The thinking lives in the agents; the honesty lives in the harness.
- **Three adversarial roles.** A **Planner** sets direction and decides when a project is done. An **Engineer** writes code, runs experiments, profiles, and edits kernels. A **Reviewer** plays the adversary — prompted to find holes and demand bolder moves, not to rubber-stamp; it is the project's sole completion authority. Each role runs its own LLM; none vouches for another.
- **Five vertical plugins.** The same harness + three roles swaps a `vertical` plugin to change domains: `research` (write a paper), `nanochat` (5-minute pretraining), `speedrun` (race a wall clock), `kernelbench` (GPU kernels). The win is defined separately for each and never cross-contaminated.

∼100k lines of Python, 137 tests, CI on every change.

2.1 Self-evolution that cannot touch the scorer

The riskiest idea in autonomous research is letting the system improve itself. Argus does, but under one non-negotiable invariant: **the frozen outcome kernel** — the metric, the validity test,

a kernel-measured wall clock, the scorer, the val data — **stays immutable and outside the agent.** The meta-loop may change the agent's *process*; it can never change what counts as winning. Self-improvement is safe *only* because ownership of the outcome stays external.

It runs as five add-only steps: a **frozen-verifier seal** (hash-pins the scorer, bounds the recipe's self-reported train-time by an independent clock); a read-only **instrumenter**; a read-only **meta-critic** shadow auditor that diagnoses the agent's own scaffolding and *applies nothing*; an **offline replay** that scores a proposed scaffolding edit at zero GPU cost; and an **asymmetric apply** path where add-only house-rules go live but any edit touching a contract is queued for human A/B and any edit touching the sealed verifier is hard-refused. Three runtime signals (a missing tool, a recurring misconfiguration, a distilled process lesson) only ever *journal advisories* — they observe, they do not auto-apply. Online A/B and framework-floor changes are deliberately left human-gated, because causal attribution at a handful of expensive trajectories is genuinely unsolved.

3 Case study: can it *invent*, or only tune?

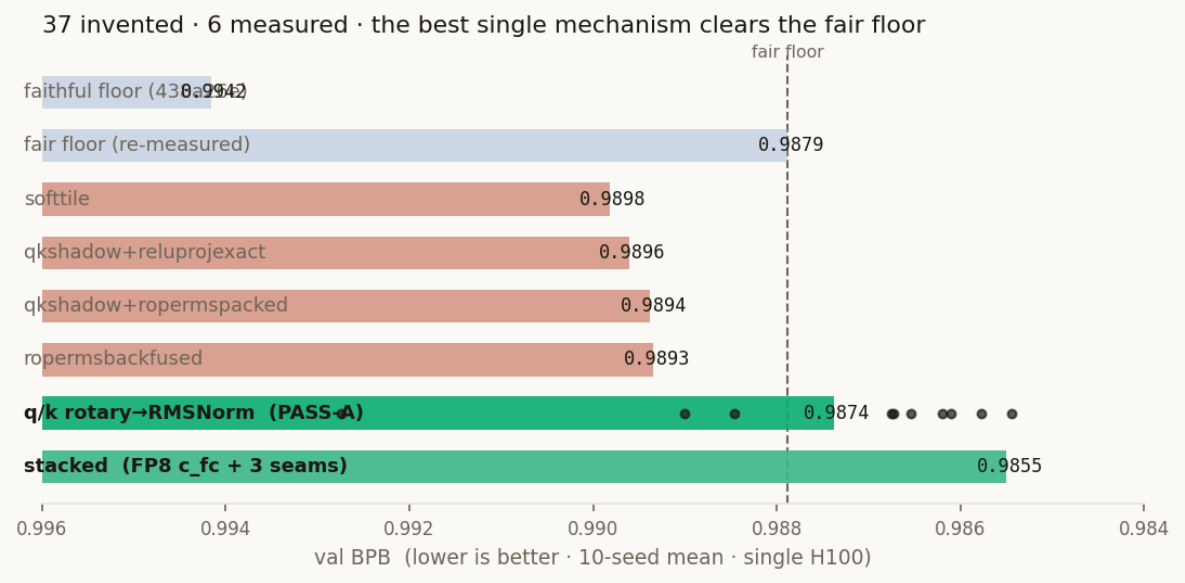
The sharpest question for any automated researcher is whether it invents new mechanisms or just recovers and tunes public recipes. We made Argus answer it under a hostile protocol.

Setup. The task is Karpathy's `autoresearch` nanochat [1]: lowest validation bits-per-byte in a fixed 5-minute, single-GPU budget, starting from the published `438a26e` baseline (MuonAdamW + value embeddings + sliding-window attention + FA3, a lineage shared with `modded-nanogpt` [2]). We imposed an **anti-cheat firewall** — verbatim from the vertical's banner:

> *"You MUST NOT search for, open, or copy the ANSWER to THIS task — the modded-nanogpt leaderboard or any published 'best' / 'optimized' solution to this exact speedrun. General method = ALLOWED; this task's published solution = DISQUALIFYING."*

A prior tuned port (`quasar`, 0.9644) was quarantined as an illegal starting point.

What happened. Over ∼3 days and 60 missions, Argus profiled the backbone, then invented and measured **37 mechanisms** — fused linear cross-entropy kernels, FP8 precision paths, persistent-shadow gradient variants, fused RoPE/RMSNorm seams — writing a postmortem for each. Six were taken to a full 10-seed measurement.



The honest result. The best *single-mechanism* invention fuses the separate `apply_rotary_emb` and `rms_norm` on query/key into one Triton kernel — computing the rotary projection in place, then normalizing on the running RMS, with a matched custom backward that reconstructs the post-rotation state instead of storing it:

```
y1 = x1 * cos + x2 * sin # rotary, in place
y2 = x2 * cos - x1 * sin
inv_rms = tl.rsqrt(tl.sum(y1*y1 + y2*y2, 1)/head_dim + eps)
tl.store(out1, y1 * inv_rms[:, None], mask=mask) # fused: normalize, no extra HBM store
tl.store(out2, y2 * inv_rms[:, None], mask=mask)
```

It reached **0.98738** (10-seed mean, 95% CI ±0.0014), edging below the same-protocol re-measured fair floor of **0.98788**. That margin, −0.0005, is *inside the confidence interval*: an honest, marginal single-mechanism win. The genuinely convincing line, **0.98550** (−0.0024), comes from *stacking* three seams, which is no longer a clean controlled delta — and we label it as such.

Two caveats we refuse to bury. First, hardware: the same baseline scores 0.9646 on a B200 and ∼0.99 on our H100 — a ∼0.029 BPB penalty that is pure hardware (less compute fits in 5 wall-clock minutes), not model quality. We do not compare our H100 numbers to anyone's B200 numbers. Second, lineage: an *earlier* Argus invention — a structured-orthogonal initializer — honestly closed as **PASS-B**, a measured negative that did not beat its own backbone. The jump from "timid initializer that loses" to "37 bold structural mechanisms, one of which wins" is the real story: a system learning to invent, not a system that always could.

4 Case study: a paper, end to end

Argus's `research` vertical runs eight stages autonomously — `research` → `plan` → `benchmark` → `run` → `analysis` → `draft` → `review` → `submission` — and has shipped two complete papers (an 11-page and a 15-page PDF, with figures, citations, and bibliographies, zero unresolved TODOs).

This is genuinely "topic to PDF with no human in the loop" — but here too the honest boundary

matters. The seven structural gates certify *form*: page count, citations, no overfull boxes, figures that resolve. **They do not certify that the contribution is true or novel.** Our own paper's abstract admits its comparator anchors are mixed (four rows within tolerance, one with no faithful published baseline, four anomalous), and the benchmark is synthetically rendered. "Produces a complete, compilable paper" is demonstrably *yes*. "Produces a paper a top venue accepts on merit" is *not something the system certifies* — and we built the gates so they cannot pretend otherwise.

5 Honesty is the architecture, not the disclaimer

The recurring theme above is not modesty for its own sake. It is the design:

- **The win is signed externally.** Because the scorer is frozen and outside the agent, the agent can be made relentlessly ambitious *and* kept honest — it cannot reward-hack a number it does not own. This is the opposite of self-rewarding setups, and it is deliberate.
- **Anti-cheat is enforced, not requested.** "Don't read the answer" is a disqualifying gate with quarantined artifacts, not a polite instruction.
- **Hardware honesty is a gate.** A wall-clock-budgeted metric is only comparable on identical silicon; cross-hardware comparisons are flagged.
- **The audit is part of the method.** A negative, measured and documented (PASS-B), is a first-class outcome — not a failure to be hidden.

On reward hacking. As search gets stronger, the binding constraint stops being capability and becomes the arms race between the agent and its own evaluator. Our answer is structural rather than vigilant: the verifier is frozen and external by construction, so the cheapest paths — reading the published answer, transplanting a recovered recipe, netting across hardware, counting a self-made benchmark as ground truth — are closed before the agent searches, not caught afterward. What this buys: nobody can refute Argus's results by catching it exaggerating. What it costs: the headline numbers are smaller than a system willing to take those shortcuts.

6 What's next

- **External validation.** We are integrating Argus into NatureBench, a third-party benchmark of 90 tasks distilled from Nature-family papers with an information firewall — the same "discover, don't reproduce" constraint Argus already enforces. A result there turns "self-reported" into "externally checkable."
- **Across more domains.** The same harness is being pointed at quantitative-finance factor research, where the frozen verifier is an out-of-sample backtest — a domain where the honesty discipline (no lookahead, IS vs OOS reported separately) is the whole game.
- **Open source.** The framework hardening for a clean third-party run is in progress.

Reproducibility

All nanochat numbers: single H100 NVL, 5-minute (298s) wall-clock budget, 10 seeds, validation BPB via the unmodified `lib.py` harness, no cherry-picking. The fair floor was re-measured under the identical shared-GPU protocol rather than reused from a different-hardware run. Live status and the full benchmark report are at argusbot.cn.

References

[1] A. Karpathy. *nanochat / autoresearch* (single-GPU, 5-min, val BPB benchmark). https://github.com/karpathy/autoresearch

[2] K. Jordan et al. *modded-nanogpt* (the NanoGPT speedrun leaderboard; Muon, FA3, record #83). https://github.com/KellerJordan/modded-nanogpt

"The harness is never smarter than the agent; the definition of winning is always external."